39.2 子 Agent

> **生成模型**：Claude Opus 4.6 (anthropic/claude-opus-4-6)
> **Token 消耗**：输入 ~300k tokens，输出 ~8k tokens（本节）

上一节介绍了多 Agent 的静态配置与解析。本节深入探讨**子 Agent（Sub-agent）** 机制——即一个 Agent 在运行时动态派生另一个 Agent 来执行特定任务的能力。这是 OpenClaw 多 Agent 架构中最具动态性的部分。

> **衍生解释：子 Agent 模式**
> 子 Agent 模式源自分布式计算中的 **Actor 模型**——每个 Actor（Agent）是独立的计算单元，通过消息传递进行通信。子 Agent 模式将其应用于 AI Agent 领域：主 Agent 像"经理"一样分配任务给专门的子 Agent，子 Agent 完成后汇报结果。这类似于操作系统中的 `fork + wait` 语义，但在更高的语义层面运作。

39.2.1 子 Agent 注册表

子 Agent 的生命周期管理由 `subagent-registry.ts` 实现。它维护了所有活跃和已归档的子 Agent 运行记录。

SubagentRunRecord 数据结构

```
// src/agents/subagent-registry.ts（简化）

type SubagentRunRecord = {
  runId: string;              // 全局唯一运行 ID（UUID）
  childSessionKey: string;    // 子 Agent 会话键
  requesterSessionKey: string; // 发起者会话键
  task: string;               // 任务描述
  cleanup: "delete" | "keep"; // 完成后的清理策略
  label?: string;             // 人类可读标签
  createdAt: number;          // 创建时间戳（ms）
  startedAt?: number;         // 实际开始时间
  endedAt?: number;           // 结束时间
  outcome?: "ok" | "error" | "timeout"; // 最终状态
  archiveAtMs?: number;       // 归档时间点
};
```

每个子 Agent 运行都有完整的生命周期追踪。 `cleanup` 字段决定了任务完成后的行为： `"delete"` 会自动清除子 Agent 的会话记录（包括转录文件）， `"keep"` 则保留以供后续审查。

注册与生命周期监听

```
const RUNS = new Map<string, SubagentRunRecord>();
const ARCHIVED: SubagentRunRecord[] = [];

export function registerSubagentRun(record: SubagentRunRecord) {
  RUNS.set(record.runId, record);
  persistSubagentRuns();    // 异步持久化到磁盘
  startLifecycleListener(); // 确保监听器已启动
}
```

注册后，系统立即将记录持久化到磁盘，并确保生命周期监听器处于活跃状态。

生命周期事件流

子 Agent 的状态变更通过 Gateway 的事件流（Server-Sent Events）进行追踪：

```
async function startLifecycleListener() {
  if (listenerActive) return;
  listenerActive = true;

  // 订阅 Gateway 的 lifecycle 事件流
  const stream = await callGateway({
    method: "events.subscribe",
    params: {
      stream: "lifecycle",
      filter: { /* 匹配子 Agent 会话 */ }
    },
  });

  // 处理三种生命周期阶段
  for await (const event of stream) {
    const { phase, sessionKey, runId } = event;
    const record = RUNS.get(runId);
    if (!record) continue;

    switch (phase) {
      case "start":
        record.startedAt = Date.now();
        break;
      case "end":
        record.endedAt = Date.now();
        record.outcome = "ok";
        scheduleArchive(record);
        break;
      case "error":
        record.endedAt = Date.now();
        record.outcome = "error";
        scheduleArchive(record);
        break;
    }
    persistSubagentRuns();
  }
}
```

生命周期事件通过 Gateway 的 WebSocket 事件系统广播，包含三个阶段： `start` （Agent 开始处理）、 `end` （正常完成）、 `error` （异常终止）。

清扫器（Sweeper）

后台清扫器以 60 秒间隔运行，负责归档和清理过期的子 Agent 记录：

```
const SWEEP_INTERVAL_MS = 60_000;

function startSweeper() {
  setInterval(() => {
    const now = Date.now();
    for (const [runId, record] of RUNS) {
      if (record.archiveAtMs && now >= record.archiveAtMs) {
        RUNS.delete(runId);
        ARCHIVED.push(record);
        finalizeSubagentCleanup(record); // 执行 delete 或 keep
      }
    }
    // 限制归档列表大小
    while (ARCHIVED.length > 100) {
      ARCHIVED.shift();
    }
    persistSubagentRuns();
  }, SWEEP_INTERVAL_MS);
}
```

`archiveAtMs` 默认设置为 `endedAt + archiveAfterMinutes * 60_000`，其中 `archiveAfterMinutes` 默认为 60 分钟。这意味着子 Agent 完成后，其记录会在内存中保留一小时（供查询和调试），然后被归档。

等待子 Agent 完成

主 Agent 可以同步等待子 Agent 完成：

```
export async function waitForSubagentCompletion(params: {
  runId: string;
  timeoutMs: number;
}): Promise<{ status: string; startedAt?: number; endedAt?: number }> {
  return callGateway({
    method: "agent.wait",
    params: {
      runId: params.runId,
      timeoutMs: params.timeoutMs,
    },
    timeoutMs: params.timeoutMs + 2000,
  });
}
```

`agent.wait` 是一个阻塞式 RPC 调用——Gateway 会挂起这个请求，直到目标子 Agent 完成或超时。注意 `timeoutMs + 2000` 的设计：Gateway 侧的超时比请求参数多 2 秒，避免请求本身因网络延迟而提前超时。

磁盘持久化与恢复

子 Agent 记录持久化到磁盘，确保 Gateway 重启后能恢复：

```
function persistSubagentRuns() {
  const data = {
    runs: Object.fromEntries(RUNS),
    archived: ARCHIVED.slice(-50), // 只保留最近 50 条归档记录
  };
  fs.writeFileSync(PERSIST_PATH, JSON.stringify(data));
}

export function restoreSubagentRunsOnce() {
  if (restored) return;
  restored = true;
  const raw = fs.readFileSync(PERSIST_PATH, "utf-8");
  const data = JSON.parse(raw);
  for (const [runId, record] of Object.entries(data.runs)) {
    RUNS.set(runId, record as SubagentRunRecord);
  }
}
```

`restoreSubagentRunsOnce()` 使用模块级 `restored` 标志确保只恢复一次。归档记录只保留最近 50 条，防止磁盘文件无限增长。

39.2.2 sessions_spawn 工具

子 Agent 的创建通过 `sessions_spawn` 工具完成。这是一个暴露给 AI Agent 的工具，允许 Agent 在对话中动态派生子 Agent。

工具参数

```
// src/agents/tools/sessions-spawn-tool.ts（简化）

const TOOL_PARAMS = {
  task: { type: "string", description: "任务描述" },
  label: { type: "string", description: "人类可读标签" },
  agentId: { type: "string", description: "目标 Agent ID" },
  model: { type: "string", description: "覆盖模型" },
  thinking: { type: "string", description: "思维级别" },
  runTimeoutSeconds: { type: "number", description: "超时（秒）" },
  cleanup: {
    type: "string",
    enum: ["delete", "keep"],
    description: "完成后是否删除会话"
  },
};
```

跨 Agent 访问控制

并非所有 Agent 都可以派生任意其他 Agent。访问控制通过 `subagents.allowAgents` 配置实现：

```
// 检查当前 Agent 是否允许派生目标 Agent
const agentConfig = resolveAgentConfig(cfg, currentAgentId);
const allowList = agentConfig?.subagents?.allowAgents;

if (allowList) {
  const isWildcard = allowList.includes("*");
  if (!isWildcard && !allowList.includes(targetAgentId)) {
    throw new Error(
      `Agent "${currentAgentId}" is not allowed to spawn "${targetAgentId}"`
    );
  }
}
```

安全规则：

配置	行为
`allowAgents: ["*"]`	允许派生任何 Agent
`allowAgents: ["coder", "researcher"]`	只允许派生指定的 Agent
`allowAgents` 未设置	使用全局默认策略

此外，系统通过 `isSubagentSessionKey()` 检查防止**子 Agent 嵌套**——子 Agent 不能再派生自己的子 Agent，避免无限递归：

```
if (isSubagentSessionKey(currentSessionKey)) {
  throw new Error("Sub-agents cannot spawn further sub-agents");
}
```

会话键生成

每个子 Agent 获得一个唯一的会话键：

```
const childSessionKey =
  `agent:${targetAgentId}:subagent:${crypto.randomUUID()}`;
```

格式为 `agent:{targetAgentId}:subagent:{uuid}` 。这个键编码了两个关键信息：目标 Agent 的 ID 和一个全局唯一标识符。Gateway 通过解析这个键来路由子 Agent 的请求到正确的 Agent 配置。

模型解析链

子 Agent 的模型通过三级优先级链解析：

```
显式参数（tool 调用时传入的 model）
    ↓（未指定）
Agent 级配置（agents.list[].subagents.model）
    ↓（未指定）
全局默认（defaults.subagents.model）
    ↓（未指定）
Agent 自身的主模型
```

解析后通过 `sessions.patch` RPC 应用到子 Agent 会话：

```
if (resolvedModel) {
  await callGateway({
    method: "sessions.patch",
    params: {
      key: childSessionKey,
      model: resolvedModel,
      thinking: normalizedThinking,
    },
  });
}
```

思维级别标准化

`thinking` 参数支持多种输入格式并统一标准化：

```
// "high" → "high", "enabled" → "medium", "off" → undefined
function normalizeThinkingLevel(raw?: string) {
  if (!raw) return undefined;
  const normalized = raw.trim().toLowerCase();
  if (normalized === "off" || normalized === "none") return undefined;
  if (normalized === "enabled" || normalized === "on") return "medium";
  return normalized; // "low", "medium", "high"
}
```

子 Agent 系统提示

每个子 Agent 在启动时注入一段特殊的系统提示词，明确其角色和约束：

```
export function buildSubagentSystemPrompt(params: {
  requesterSessionKey?: string;
  requesterOrigin?: DeliveryContext;
  childSessionKey: string;
  label?: string;
  task?: string;
}) {
  const lines = [
    "# Subagent Context",
    "",
    "You are a **subagent** spawned by the main agent.",
    "",
    "## Your Role",
    `- You were created to handle: ${taskText}`,
    "- Complete this task. That's your entire purpose.",
    "",
    "## Rules",
    "1. **Stay focused** - Do your assigned task, nothing else",
    "2. **Complete the task** - Your final message will be reported",
    "3. **Don't initiate** - No heartbeats, no proactive actions",
    "4. **Be ephemeral** - You may be terminated after completion.",
    "",
    "## What You DON'T Do",
    "- NO user conversations",
    "- NO external messages (unless explicitly tasked)",
    "- NO cron jobs or persistent state",
    "- NO pretending to be the main agent",
    // ...
  ];
  return lines.join("\n");
}
```

这段系统提示确保子 Agent：

- 专注于分配的任务
- 不会擅自与用户交互
- 不会创建持久化副作用（如定时任务）
- 清楚自己是临时的、可被销毁的

完整的 spawn 流程

将上述组件串联起来， `sessions_spawn` 的完整执行流程如下：

```
1. 验证权限（allowAgents 检查 + 嵌套检查）
     ↓
2. 生成 childSessionKey
     ↓
3. 解析模型和思维级别
     ↓
4. sessions.patch 设置模型/thinking
     ↓
5. buildSubagentSystemPrompt 生成系统提示
     ↓
6. callGateway("agent") 启动子 Agent
     参数: lane=AGENT_LANE_SUBAGENT, deliver=false, spawnedBy=runId
     ↓
7. registerSubagentRun 注册到注册表
     ↓
8. 返回 { runId, sessionKey } 给主 Agent
```

`lane=AGENT_LANE_SUBAGENT` 告诉 Gateway 这是一个子 Agent 运行， `deliver=false` 表示子 Agent 的输出不直接投递给用户通道——而是通过公告流程（下一小节）由主 Agent 转述。

39.2.3 子 Agent 公告

子 Agent 完成任务后，其结果需要**公告**给主 Agent，再由主 Agent 以自然的方式告知用户。 `subagent-announce.ts` 实现了这一桥接机制。

公告流程概览

```
子 Agent 完成
    ↓
runSubagentAnnounceFlow()
    ↓
waitForSessionUsage() — 等待 token 统计就绪
    ↓
buildSubagentStatsLine() — 构建统计行
    ↓
构建触发消息（包含子 Agent 结果 + 统计 + 指令）
    ↓
maybeQueueSubagentAnnounce() — 决定投递方式
    ↓
┌── steered → 注入到当前活跃的 Agent 对话中
├── queued  → 放入公告队列等待合适时机
└── none    → 直接发送给主 Agent
```

等待用量统计

子 Agent 完成后，token 使用量可能尚未写入会话存储（写入是异步的）。 `waitForSessionUsage()` 通过轮询等待：

```
async function waitForSessionUsage(params: { sessionKey: string }) {
  let entry = loadSessionStore(storePath)[params.sessionKey];
  const hasTokens = () =>
    entry &&
    (typeof entry.totalTokens === "number" ||
     typeof entry.inputTokens === "number" ||
     typeof entry.outputTokens === "number");

  if (hasTokens()) return { entry, storePath };

  // 最多等待 4 × 200ms = 800ms
  for (let attempt = 0; attempt < 4; attempt++) {
    await new Promise((r) => setTimeout(r, 200));
    entry = loadSessionStore(storePath)[params.sessionKey];
    if (hasTokens()) break;
  }
  return { entry, storePath };
}
```

4 次重试、每次 200ms 间隔——总共最多等待 800ms。这是一个典型的**最终一致性**等待模式：系统不保证用量数据立即可用，但在合理时间内会收敛。

统计行构建

```
async function buildSubagentStatsLine(params: {
  sessionKey: string;
  startedAt?: number;
  endedAt?: number;
}) {
  // ...
  const parts: string[] = [];
  parts.push(`runtime ${formatDurationShort(runtimeMs) ?? "n/a"}`);
  parts.push(`tokens ${totalText} (in ${inputText} / out ${outputText})`);
  if (costText) parts.push(`est ${costText}`);
  parts.push(`sessionKey ${params.sessionKey}`);
  // ...
  return `Stats: ${parts.join(" • ")}`;
}
```

统计行包含运行时长、token 消耗、估算成本、会话键和转录文件路径。成本估算基于配置中的模型定价信息（`models.providers[provider].models[].cost`）。

格式化工具函数遵循人类可读原则：

值	`formatTokenCount`	`formatDurationShort`	`formatUsd`
1,500,000	`"1.5m"`	—	—
42,300	`"42.3k"`	—	—
185 秒	—	`"3m5s"`	—
0.0042	—	—	`"$0.0042"`
1.23	—	—	`"$1.23"`

触发消息构造

公告的核心是构造一条**触发消息**发送给主 Agent，让主 Agent 以自然语言转述结果：

```
const triggerMessage = [
  `A background task "${taskLabel}" just ${statusLabel}.`,
  "",
  "Findings:",
  reply || "(no output)",
  "",
  statsLine,
  "",
  "Summarize this naturally for the user. Keep it brief.",
  "Do not mention technical details like tokens or stats.",
  "You can respond with NO_REPLY if no announcement is needed.",
].join("\n");
```

这条消息包含了子 Agent 的完整输出，但最后的指令要求主 Agent **转译**而非直接转发。用户看到的是主 Agent 自然流畅的总结，而不是机械的技术报告。

投递策略

`maybeQueueSubagentAnnounce()` 根据当前主 Agent 的状态决定如何投递：

```
async function maybeQueueSubagentAnnounce(params) {
  const queueSettings = resolveQueueSettings({ cfg, channel, sessionEntry });
  const isActive = isEmbeddedPiRunActive(sessionId);

  // 模式 1: steer — 注入当前活跃对话
  if (shouldSteer) {
    const steered = queueEmbeddedPiMessage(sessionId, triggerMessage);
    if (steered) return "steered";
  }

  // 模式 2: queue — 放入队列等待
  if (isActive && shouldFollowup) {
    enqueueAnnounce({ key, item, settings, send });
    return "queued";
  }

  // 模式 3: none — 直接发送
  return "none";
}
```

三种投递模式：

模式	条件	行为
steer	主 Agent 正在运行 + steer 模式	将消息注入到当前 Agent 轮次中，Agent 可以在回复中自然提及
queued	主 Agent 正在运行 + followup/collect 模式	放入公告队列，等待 Agent 空闲时批量投递
none / 直接发送	主 Agent 空闲	立即发送触发消息，启动新的 Agent 轮次

清理与善后

公告流程的 `finally` 块处理善后工作：

```
finally {
  // 设置子 Agent 会话标签
  if (params.label) {
    await callGateway({
      method: "sessions.patch",
      params: { key: childSessionKey, label: params.label },
    });
  }
  // 按 cleanup 策略处理会话
  if (params.cleanup === "delete") {
    await callGateway({
      method: "sessions.delete",
      params: { key: childSessionKey, deleteTranscript: true },
    });
  }
}
```

39.2.4 公告队列

当多个子 Agent 同时完成时，它们的公告需要有序地投递给主 Agent，避免消息洪泛。 `subagent-announce-queue.ts` 实现了这一队列机制。

队列状态

```
type AnnounceQueueState = {
  items: AnnounceQueueItem[];  // 待投递的公告
  draining: boolean;           // 是否正在排水
  lastEnqueuedAt: number;      // 最后入队时间
  mode: QueueMode;             // 队列模式
  debounceMs: number;          // 防抖间隔（默认 1000ms）
  cap: number;                 // 最大容量（默认 20）
  dropPolicy: QueueDropPolicy; // 溢出策略
  droppedCount: number;        // 已丢弃计数
  summaryLines: string[];      // 丢弃消息的摘要行
  send: (item) => Promise<void>; // 发送回调
};
```

防抖与批量投递

队列采用**防抖**（debounce）策略：入队后等待 `debounceMs` （默认 1 秒），如果期间有新消息入队则重置计时器。这样，短时间内完成的多个子 Agent 的公告会被合并处理。

当队列模式为 `"collect"` 时，所有待投递的消息会被合并成一条：

```
if (queue.mode === "collect") {
  // 跨通道检测：不同通道的消息不能合并
  if (isCrossChannel) {
    // 逐条发送
    const next = queue.items.shift();
    await queue.send(next);
    continue;
  }
  // 同通道：合并所有消息
  const items = queue.items.splice(0, queue.items.length);
  const prompt = buildCollectPrompt({
    title: "[Queued announce messages while agent was busy]",
    items,
    renderItem: (item, idx) => `Queued #${idx + 1}\n${item.prompt}`,
  });
  await queue.send({ ...last, prompt });
}
```

注意**跨通道安全**：如果待投递的消息来自不同通道（如一个来自 Telegram、一个来自 Discord），系统会切换到逐条发送模式，避免将不同通道的上下文混合。

溢出策略

当队列达到上限（默认 20 条）时， `dropPolicy` 决定如何处理新消息：

策略	行为
`"summarize"`	丢弃消息但记录摘要行，最终以汇总形式告知主 Agent
`"new"`	丢弃新入队的消息（旧消息保留）
`"old"`	丢弃最旧的消息（新消息入队）

`"summarize"` 是默认策略——即使消息被丢弃，主 Agent 仍会收到类似"另有 N 条公告被汇总"的通知，确保不会完全丢失信息。

39.2.5 子 Agent 控制范围（v2026.3.9 新增）

v2026.3.9 引入了**子 Agent 能力系统**（ `src/agents/subagent-capabilities.ts` ）和**控制系统**（ `src/agents/subagent-control.ts` ），对子 Agent 的权限进行了更细粒度的管控。

角色与控制范围

每个会话根据其在子 Agent 树中的**深度**（depth）被分配一个角色：

```
// src/agents/subagent-capabilities.ts
type SubagentSessionRole = "main" | "orchestrator" | "leaf";
type SubagentControlScope = "children" | "none";

function resolveSubagentCapabilities(params: { depth: number; maxSpawnDepth?: number })
  // depth=0 → main（完全控制）
  // 0 < depth < maxSpawnDepth → orchestrator（可派生、可控制子级）
  // depth >= maxSpawnDepth → leaf（不可派生、不可控制）
  return { role, controlScope, canSpawn, canControlChildren };
}
```

角色	深度	可派生子 Agent	可控制子 Agent	说明
`main`	0	✅	✅ 全部	主会话，拥有完整控制权

角色	深度	可派生子 Agent	可控制子 Agent	说明
`orchestrator`	1..N-1	✅	✅ 仅自己派生的	中间层，可编排下级
`leaf`	≥N	❌	❌	叶子节点，只能执行任务

所有权校验

控制操作（如终止子 Agent、发送消息）前，系统会校验**所有权**——一个子 Agent 只能控制它自己派生的运行：

```
// src/agents/subagent-control.ts
function ensureControllerOwnsRun(params) {
  const owner = entry.controllerSessionKey || entry.requesterSessionKey;
  if (owner === controller.controllerSessionKey) return undefined; // 通过
  return "Subagents can only control runs spawned from their own session.";
}
```

这一设计防止了**横向权限逃逸**——即使两个子 Agent 运行在同一层级，它们也无法互相干扰。 `leaf` 节点恢复会话后不会重新获得编排权限，确保了权限模型的**单调性**（Monotonicity）。

本节小结

1. **SubagentRunRecord** 追踪子 Agent 完整生命周期：从创建到归档，包含时间戳、状态、清理策略等元数据。
2. **生命周期监听** 通过 Gateway 的 lifecycle 事件流（start/end/error）实时更新子 Agent 状态。
3. **清扫器** 以 60 秒间隔运行，归档超时记录（默认 60 分钟），限制归档列表为 100 条。
4. **sessions_spawn** 工具实现了完整的 spawn 流程：权限检查 → 会话键生成 → 模型解析 → 系统提示注入 → Gateway 调用 → 注册表记录。
5. **跨 Agent 安全**： `allowAgents` 配置控制哪些 Agent 可以被派生，子 Agent 不能嵌套（防止无限递归）。
6. **子 Agent 系统提示** 明确了子 Agent 的角色约束：专注任务、不与用户交互、不创建持久副作用。
7. **公告流程** 将子 Agent 的结果转化为自然语言，通过 steer/queue/direct 三种策略投递给主 Agent。
8. **公告队列** 使用防抖、批量合并和跨通道安全检测，防止消息洪泛，确保用户体验流畅。